

DEVSECOPS PIPELINE FOR SECURE SOFTWARE DELIVERY PLATFORM

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Faculty:Dr.Ramireddy Navatejareddy

M.Tharani(192472375)

1. Introduction

1.1 What is DevSecOps

DevSecOps extends DevOps by embedding security directly into development and delivery instead of treating it as a separate, end-of-cycle activity. Checks such as code scanning, dependency analysis, and configuration review run automatically inside the same pipeline that builds, tests, and deploys the application, so every commit is evaluated for security at the same time it is evaluated for correctness.

1.2 Why Secure Delivery Matters

A delivery pipeline moves code from a developer's machine to production. If any stage is compromised, an attacker can inject malicious code into every application that passes through it afterwards, which is why several real supply-chain attacks have targeted build systems rather than individual applications. A breach caused this way also brings financial loss, regulatory penalties, and reputational damage, so treating the pipeline itself as a security boundary is essential.

1.3 Problems in Traditional DevOps

Traditional DevOps optimises for speed but often leaves security as a manual step performed weeks after a feature has already shipped. This makes vulnerabilities expensive to fix, denies developers fast feedback on the security implications of their own code, and leaves no automated gate capable of stopping an insecure build from reaching production.

1.4 Integrating Security Across the SDLC

Security therefore needs to be present at every SDLC stage: requirements capture during planning, static analysis during coding, dependency and image scanning during the build, dynamic testing during QA, signed artefacts at deployment, and continuous monitoring in production. This turns

security into a shared team responsibility rather than a separate hand-off, and is the principle that guided the design of this platform.

1.5 Objectives of the Project

- Design a DevSecOps pipeline that automates build, test, security scanning, and deployment.
- Apply Agile practices — epics, user stories, sprint planning — to organise the work.
- Embed SAST, DAST, dependency scanning, and container scanning into the pipeline.
- Define role-based access control for platform functionality.
- Produce a realistic backlog, sprint plan, and estimation model.
- Establish monitoring and reporting for delivery progress and security posture.

2. Project Vision

2.1 Vision Statement

For teams who need to release software quickly without exposing the organisation to unnecessary risk, the Secure Software Delivery Platform is a DevSecOps-enabled CI/CD system that automatically builds, tests, scans, and deploys applications while enforcing security gates at every stage, so a build cannot reach production unless it passes them.

2.2 Objectives, Business Goals and Benefits

Project objectives: reduce vulnerability detection time from weeks to minutes; automate at least 80% of manual release checks; provide role-based dashboards; maintain a complete audit trail of every change. Business goals: shorter release cycles, lower cost of late-stage remediation, and audit evidence for frameworks such as ISO 27001 or SOC 2. Expected benefits: faster and safer releases, lower remediation cost, stronger compliance posture, and better management visibility into delivery and security metrics.

2.3 Scope and Success Criteria

The scope covers Agile planning of the pipeline — authentication, source code management, CI/CD automation, container security, vulnerability scanning, RBAC, monitoring, and reporting — while infrastructure provisioning and long-term production support are out of scope for this assessment. Success criteria: a prioritised backlog mapping every story to business value and MoSCoW category; a realistic Sprint 1 backlog broken down to the hour; a Given-When-Then acceptance criterion for every story; and at least one automated control for each SDLC stage.

3. Stakeholder Identification

Table 1 identifies the primary stakeholders, their responsibilities, and their relative interest and influence over the project.

Stakeholder	Role	Responsibilities	Interest	Influence
Administrator	Platform Owner	Configures the platform and enforces global security policy	High	High
Developer	Contributor	Writes code and fixes flagged vulnerabilities	High	Medium
Security Engineer	Security Reviewer	Defines policy, reviews scans, configures tools	High	High
DevOps Engineer	Pipeline Owner	Builds and maintains the pipeline and infrastructure	High	High
QA Engineer	Quality Reviewer	Designs and executes tests, validates staging builds	High	Medium
Project Manager	Coordinator	Plans sprints and communicates status	Medium	High
Compliance Officer	Auditor	Reviews audit trails and vulnerability reports	Medium	Medium
Client	Sponsor	Funds the project and defines requirements	High	High
End Users	Consumers	Use delivered applications	Medium	Low

Table 1: Stakeholder Analysis

Figure 1: DevSecOps Lifecycle — Plan, Code, Build, Test, Release, Deploy, Operate, Monitor, with a security gate at each stage (image placeholder)

4. Epics

Eight epics organise the work required to deliver the platform. Each is too large for a single sprint and is broken into the user stories in Section 5.

4.1 EP-01: User Authentication and Access Management

Covers registration, login, multi-factor authentication, password recovery, and credential policy. Weak authentication would undermine every other security control, so this epic is the foundation of the platform.

4.2 EP-02: Source Code Management and Repository Security

Protects the integrity of source code in version control: repository access, branch protection, mandatory review, and commit signing, since source code is the platform's most valuable asset.

4.3 EP-03: CI/CD Pipeline Automation

Automates build, test, and deployment so every change is verified consistently, removing manual error-prone steps and giving each stage a place to insert a security gate.

4.4 EP-04: Container Security and Image Hardening

Scans container images for known issues, restricts approved image sources, runs containers with minimal privileges, and signs images so only verified artefacts are deployed.

4.5 EP-05: Vulnerability and Dependency Scanning

Adds automated static analysis, dynamic testing, and dependency scanning into the pipeline so security issues are caught as early as possible, the central idea behind DevSecOps.

4.6 EP-06: Role Based Access Control and Policy Enforcement

Ensures authenticated users can only perform actions appropriate to their role, covering permission assignment, periodic review, audit trails, and approval workflows for sensitive access.

4.7 EP-07: Monitoring, Logging and Alerting

Provides visibility into the pipeline and running application through centralised logs, real-time alerts, and health dashboards so incidents are detected before they cause damage.

4.8 EP-08: Reporting and Compliance Dashboard

Generates reports summarising pipeline activity, security posture, and sprint progress for managers, compliance officers, and clients, turning raw pipeline data into decisions.

5. User Stories

Thirty user stories break the epics into plannable work, each written as "As a <role>, I want <feature>, so that <benefit>" and grouped by epic.

5.1 User Authentication and Access Management

- US-01: As a registered user, I want to log in using multi-factor authentication, so that my account remains protected against unauthorized access.
- US-02: As a new user, I want to register for an account using my organisational email, so that I can access the platform securely.
- US-03: As a user, I want to reset my forgotten password through a secure verification link, so that I can regain access without compromising security.
- US-04: As an administrator, I want to enforce password complexity policies, so that weak credentials do not become an attack vector.

5.2 Source Code Management and Repository Security

- US-05: As a developer, I want to push code to a version-controlled repository, so that all changes are tracked and auditable.
- US-06: As a developer, I want branch protection rules enforced on the main branch, so that unreviewed code cannot be merged into production.
- US-07: As a security engineer, I want commit signing to be mandatory, so that code authenticity can be verified.
- US-08: As a developer, I want automated code review checks before merging, so that code quality and security standards are maintained.

5.3 CI/CD Pipeline Automation

- US-09: As a DevOps engineer, I want an automated build pipeline triggered on every commit, so that integration issues are detected early.
- US-10: As a DevOps engineer, I want automated unit and integration tests to run in the pipeline, so that regressions are caught before deployment.
- US-11: As a DevOps engineer, I want the pipeline to automatically deploy to a staging environment, so that changes can be validated before production release.
- US-12: As a project manager, I want pipeline execution status visible on a dashboard, so that the team can track build health in real time.

5.4 Container Security and Image Hardening

- US-13: As a DevOps engineer, I want container images scanned for vulnerabilities before being pushed to the registry, so that insecure images are never deployed.
- US-14: As a security engineer, I want base images restricted to an approved internal registry, so that untrusted images cannot be used.
- US-15: As a DevOps engineer, I want containers to run with least-privilege permissions, so that a compromised container cannot escalate access.
- US-16: As a security engineer, I want image signing enforced before deployment, so that only verified images run in production.

5.5 Vulnerability and Dependency Scanning

- US-17: As a security engineer, I want static application security testing (SAST) integrated into the pipeline, so that code-level vulnerabilities are identified early.
- US-18: As a security engineer, I want dynamic application security testing (DAST) performed on staging builds, so that runtime vulnerabilities are detected.
- US-19: As a developer, I want dependency scanning to flag vulnerable open-source libraries, so that known CVEs are addressed before release.
- US-20: As a compliance officer, I want a consolidated vulnerability report generated after every scan, so that risk exposure is documented for audits.

5.6 Role Based Access Control and Policy Enforcement

- US-21: As a administrator, I want to assign role-based permissions to users, so that access is limited to what each role requires.
- US-22: As a administrator, I want to review and revoke access for inactive users, so that orphaned accounts do not pose a risk.
- US-23: As a compliance officer, I want an audit trail of all permission changes, so that access modifications can be reviewed.
- US-24: As a project manager, I want approval workflows for elevated access requests, so that privilege escalation is controlled.

5.7 Monitoring, Logging and Alerting

- US-25: As a DevOps engineer, I want centralised logging of pipeline and application events, so that issues can be diagnosed quickly.

- US-26: As a security engineer, I want real-time alerts for suspicious activity, so that incidents can be responded to promptly.
- US-27: As a DevOps engineer, I want system health dashboards for infrastructure metrics, so that performance degradation is detected early.

5.8 Reporting and Compliance Dashboard

- US-28: As a project manager, I want sprint progress reports generated automatically, so that stakeholders remain informed without manual effort.
- US-29: As a compliance officer, I want compliance status reports mapped to security standards, so that audit readiness can be demonstrated.
- US-30: As a client, I want a summary dashboard of delivery metrics, so that I can track project value and quality trends.

6. Acceptance Criteria

Every story has an acceptance criterion in Given-When-Then format, listed below in story order.

ID	Given	When	Then
US-01	a registered user has entered a valid username and password	the system prompts for a one-time code sent to the registered device	access is granted only after the correct code is entered within the valid time window
US-02	a new user submits the registration form with an organisational email address	the system validates the domain and sends a confirmation link	the account is activated only after the user confirms the email
US-03	a user has requested a password reset	they click the secure link sent to their registered email within its validity period	they are allowed to set a new password that meets complexity rules
US-04	an administrator has configured the password policy	a user attempts to set a password that does not meet the minimum complexity rules	the system rejects the password and displays the specific rule that was not met
US-05	a developer has made local changes	they push the commit to the remote repository	the commit is recorded with author, timestamp, and a unique hash for traceability

ID	Given	When	Then
US-06	a pull request targets the main branch	the required number of approvals has not been met	the merge button remains disabled until the review requirement is satisfied
US-07	commit signing is enforced on the repository	a developer pushes an unsigned commit	the push is rejected and the developer is informed that a GPG signature is required
US-08	a pull request has been opened	the automated review checks detect a coding standard violation	the pipeline marks the check as failed and blocks the merge until it is resolved
US-09	a developer pushes a commit to any tracked branch	the commit is received by the version control system	the CI server automatically starts a new build within a defined time limit
US-10	a build has started	the test stage executes the unit and integration test suite	the pipeline proceeds only if all tests pass, otherwise it halts and reports the failing tests
US-11	all pipeline test stages have passed	the deployment stage is triggered	the build artefact is automatically deployed to the staging environment without manual steps
US-12	a pipeline run is in progress or has completed	a project manager opens the dashboard	the current status, duration, and outcome of each stage are displayed in real time
US-13	a container image has been built	it is scanned against a known vulnerability database	the image is blocked from being pushed to the registry if a critical vulnerability is found
US-14	a Dockerfile references a base image	the image source is checked during the build	the build fails if the base image does not come from the approved internal registry

ID	Given	When	Then
US-1 5	a container is scheduled to run	the runtime configuration is applied	the container starts with only the permissions explicitly required by the application
US-1 6	an image has passed all required security checks	it is prepared for deployment	the image is cryptographically signed and only signed images are accepted by the deployment stage
US-1 7	source code is pushed to the repository	the SAST stage of the pipeline runs	the pipeline reports any identified code-level vulnerabilities with severity and location
US-1 8	an application is running in the staging environment	the DAST tool executes automated attack simulations	identified runtime vulnerabilities are logged with a severity rating for triage
US-1 9	a project declares third-party dependencies	the dependency scanning stage runs	any dependency with a known CVE is flagged and the pipeline reports the affected package and version
US-2 0	one or more security scans have completed	the reporting job runs at the end of the pipeline	a single consolidated vulnerability report is generated and stored for audit purposes
US-2 1	an administrator is managing user accounts	they assign a role to a user	the user is granted only the permissions defined for that role
US-2 2	a user account has shown no activity for a defined period	the periodic access review job runs	the account is flagged for review and access is

ID	Given	When	Then
			suspended pending administrator confirmation
US-2 3	a permission change has been made to any account	the change is committed	an entry is recorded in the audit log with who made the change, what changed, and when
US-2 4	a user requests elevated access	the request is submitted	the request is routed to an authorised approver and access is granted only after explicit approval
US-2 5	an event occurs in the pipeline or the application	the event is generated	it is forwarded to the centralised logging system with a timestamp and source identifier
US-2 6	the monitoring system detects a pattern matching suspicious activity	the threshold for that pattern is exceeded	an alert is sent to the security team within a defined response time
US-2 7	infrastructure metrics are being collected	a DevOps engineer opens the health dashboard	current CPU, memory, and availability metrics are displayed with historical trends
US-2 8	a sprint is in progress	the reporting job runs at the end of each day	a progress report showing completed, in-progress, and pending stories is generated automatically
US-2 9	compliance requirements have been mapped to pipeline controls	a compliance officer requests a status report	the report shows which controls are satisfied and which require attention
US-3 0	delivery metrics have been collected across sprints	a client opens the summary dashboard	key metrics such as velocity, defect rate, and delivery

ID	Given	When	Then
			dates are displayed in an easy-to-read format

Table 2: Acceptance Criteria for All User Stories

7. Product Backlog

Priority combines MoSCoW category and business value; story points use the Fibonacci scale from Section 9. Table 3 shows the full backlog sorted by descending priority.

Backlog ID	Epic	Story	Bus. Value	Priority	Dependency	MoSCoW	Points
PB-01	EP-01	US-01	High	1	None	Must	5
PB-02	EP-01	US-02	High	2	None	Must	3
PB-03	EP-01	US-04	High	3	US-02	Must	2
PB-04	EP-02	US-05	High	4	None	Must	3
PB-05	EP-02	US-06	High	5	US-05	Must	5
PB-06	EP-03	US-09	High	6	US-05	Must	5
PB-07	EP-03	US-10	High	7	US-09	Must	8
PB-08	EP-04	US-13	High	8	US-09	Must	8
PB-09	EP-05	US-17	High	9	US-09	Must	8
PB-10	EP-05	US-19	High	10	US-09	Must	5

Backlog ID	Epic	Story	Bus. Value	Priority	Dependency	MoSCoW	Points
PB-11	EP-06	US-21	High	11	US-02	Must	3
PB-12	EP-07	US-25	High	12	US-09	Must	5
PB-13	EP-01	US-03	Medium	13	US-02	Should	3
PB-14	EP-02	US-07	Medium	14	US-05	Should	3
PB-15	EP-02	US-08	Medium	15	US-06	Should	5
PB-16	EP-03	US-11	Medium	16	US-10	Should	5
PB-17	EP-04	US-14	Medium	17	US-13	Should	5
PB-18	EP-04	US-15	Medium	18	US-13	Should	5
PB-19	EP-05	US-18	Medium	19	US-11	Should	8
PB-20	EP-06	US-22	Medium	20	US-21	Should	2
PB-21	EP-06	US-23	Medium	21	US-21	Should	3
PB-22	EP-07	US-26	Medium	22	US-25	Should	5
PB-23	EP-08	US-29	Medium	23	US-20	Should	3
PB-24	EP-03	US-12	Medium	24	US-09	Could	3

Backlog ID	Epic	Story	Bus. Value	Priority	Dependency	MoSCoW	Points
PB-25	EP-04	US-16	Medium	25	US-14	Could	5
PB-26	EP-05	US-20	Medium	26	US-17	Could	3
PB-27	EP-08	US-28	Medium	27	None	Could	2
PB-28	EP-06	US-24	Low	28	US-21	Could	5
PB-29	EP-07	US-27	Low	29	US-25	Could	3
PB-30	EP-08	US-30	Low	30	US-12	Won't	3

Table 3: Product Backlog (sorted by priority)

8. Sprint Planning

8.1 Team Composition and Capacity

Sprint 1 runs for two weeks (ten working days) with one Scrum Master, three Developers, one QA Engineer, and one DevOps Engineer. The Scrum Master facilitates rather than delivers points, so capacity is drawn from the other five members at roughly six productive hours per day, giving about 300 hours, or 48–50 story points at an initial planning ratio of six hours per point.

Role	Count	Calculation	Hours
Scrum Master	1	Facilitation only	0
Developer	3	10 days $\times$ 6 hrs	180
QA Engineer	1	10 days $\times$ 6 hrs	60
DevOps Engineer	1	10 days $\times$ 6 hrs	60

Table 4: Sprint Capacity Calculation

8.2 Sprint Backlog

Sprint 1 selects 9 stories totalling 48 points, fitting within capacity while respecting dependencies (e.g. US-06 requires US-05 first).

Story ID	Epic	MoSCoW	Value	Points	Dependency
US-01	User Authentication and Access Management	Must	High	5	None
US-02	User Authentication and Access Management	Must	High	3	None
US-05	Source Code Management and Repository Security	Must	High	3	None
US-06	Source Code Management and Repository Security	Must	High	5	US-05
US-09	CI/CD Pipeline Automation	Must	High	5	US-05
US-10	CI/CD Pipeline Automation	Must	High	8	US-09
US-13	Container Security and Image Hardening	Must	High	8	US-09
US-17	Vulnerability and Dependency Scanning	Must	High	8	US-09
US-21	Role Based Access Control and Policy Enforcement	Must	High	3	US-02

Table 5: Sprint 1 Backlog

8.3 Task Breakdown

Each Sprint 1 story is split into three tasks — implementation, testing, and review — distributed across the three Developers, the QA Engineer, and the DevOps Engineer.

Task ID	Task	Owner	Hours	Status
T-1	US-01: Implementation	Developer 1	20	To Do
T-2	US-01: Testing	QA Engineer	8	To Do
T-3	US-01: Review & documentation	Developer 2	2	To Do

Task ID	Task	Owner	Hours	Status
T-4	US-02: Implementation	Developer 2	12	To Do
T-5	US-02: Testing	QA Engineer	5	To Do
T-6	US-02: Review & documentation	Developer 3	1	To Do
T-7	US-05: Implementation	Developer 3	12	To Do
T-8	US-05: Testing	QA Engineer	5	To Do
T-9	US-05: Review & documentation	Developer 1	1	To Do
T-10	US-06: Implementation	Developer 1	20	To Do
T-11	US-06: Testing	QA Engineer	8	To Do
T-12	US-06: Review & documentation	Developer 2	2	To Do
T-13	US-09: Implementation	Developer 2	20	To Do
T-14	US-09: Testing	QA Engineer	8	To Do
T-15	US-09: Review & documentation	Developer 3	2	To Do
T-16	US-10: Implementation	Developer 3	31	To Do
T-17	US-10: Testing	QA Engineer	12	To Do
T-18	US-10: Review & documentation	Developer 1	5	To Do
T-19	US-13: Implementation	Developer 1	31	To Do
T-20	US-13: Testing	QA Engineer	12	To Do
T-21	US-13: Review & documentation	Developer 2	5	To Do
T-22	US-17: Implementation	Developer 2	31	To Do
T-23	US-17: Testing	QA Engineer	12	To Do
T-24	US-17: Review & documentation	Developer 3	5	To Do
T-25	US-21: Implementation	Developer 3	12	To Do
T-26	US-21: Testing	QA Engineer	5	To Do
T-27	US-21: Review & documentation	Developer 1	1	To Do

Table 6: Sprint 1 Task Breakdown

9. Story Point Estimation

Story points use the Fibonacci scale (1, 2, 3, 5, 8, 13), whose widening gaps reflect the growing uncertainty of larger tasks. Estimation was carried out using Planning Poker during backlog refinement so no single voice anchors the group's estimate.

Points	Meaning in this Project
1–2	Trivial to small, well-understood change with minimal testing.
3	Small feature with a couple of moving parts.
5	Medium feature spanning more than one component or story.
8	Large feature with real technical or security uncertainty.
13	Very large item that should usually be split before a sprint.

Table 7: Fibonacci Scale Definitions

10. Sprint Goal

"By the end of Sprint 1, users can securely register and authenticate, developers can push code through a branch-protected repository, and every commit automatically triggers a pipeline that tests, scans for vulnerabilities, and checks container images before anything is deployed."

This goal can be verified without ambiguity: at the Sprint Review the team demonstrates a live login, a protected branch rejecting an unreviewed pull request, and a pipeline run with a passing security scan.

11. Expected Deliverables

Table 8 lists the working deliverables demonstrated at the Sprint 1 Review, each mapped back to its source stories.

Deliverable	Description
Authentication Module	Login with MFA and registration (US-01, US-02)
CI/CD Pipeline Setup	Automated build with branch protection (US-05, US-06, US-09)
Basic Security Scan	Automated tests, image scan, and SAST (US-10, US-13, US-17)
Dashboard	Basic pipeline run status view (progress toward US-12)
User Management	Role assignment for Administrators (US-21)

Deliverable	Description
Documentation	Updated README, pipeline configuration notes, and this report

Table 8: Sprint 1 Deliverables

12. Definition of Done

The Definition of Done is a shared checklist ensuring "done" means the same thing to every role, including security checks rather than functional completion alone:

- Code committed to the correct branch and peer-reviewed and approved.
- All unit and integration tests pass in the pipeline.
- SAST has run with no unresolved critical or high findings.
- Any container image has passed the vulnerability scan.
- Acceptance criteria verified by the QA Engineer.
- New configuration or dependencies are documented.
- Change deployed successfully to staging and accepted by the Product Owner.

13. Risk Analysis

Table 9 lists the risks considered most relevant to this project.

Risk	Impact	Probability	Mitigation
Scanning tools produce many false positives, slowing the pipeline	High	Medium	Tune rules early; maintain a reviewed exceptions list
Key DevOps Engineer becomes unavailable	High	Low	Cross-train a Developer; document pipeline scripts
Container scanning integration underestimated	Medium	Medium	Timebox a spike; split the story if needed
Scope creep from stakeholder requests	Medium	High	Route requests through the Product Owner for backlog triage
Branch protection blocks emergency hotfixes	Medium	Low	Define an audited emergency override process

Risk	Impact	Probability	Mitigation
Regulatory gap found late in the project	High	Low	Involve the Compliance Officer from Sprint 1 onward

Table 9: Risk Management Table

14. Agile Ceremonies

Five ceremonies keep the team synchronised around the two-week sprint cycle. Sprint Planning selects stories and breaks them into tasks. The Daily Scrum is a fifteen-minute daily check-in on progress and blockers, including failing security scans. Sprint Review demonstrates working software, such as the authentication module and a passing pipeline run, to stakeholders. Sprint Retrospective looks inward at the team's own process, including whether security gates are slowing delivery. Backlog Refinement, held mid-sprint, keeps upcoming items estimated and ready to pull into the next Sprint Planning session.

15. Conclusion

This report worked through the full Agile planning cycle for a DevSecOps Pipeline for a Secure Software Delivery Platform: vision and stakeholder analysis, eight epics and thirty user stories, and an estimated, task-broken Sprint 1 backlog. Security was written directly into the epics, stories, and Definition of Done rather than bolted on afterward, which reflects how DevSecOps works in practice. Breaking security requirements into small, testable increments and verifying them with Given-When-Then acceptance criteria showed that security automation carries a real, budgetable cost in points and hours — and that Agile sprint planning gives a team a realistic path to building a platform that is both fast and secure.

JIRA PROJECT REPOSITORY

The Agile planning and project management activities for the DevSecOps Pipeline for Secure Software Delivery Platform were implemented using Atlassian Jira. The Jira project includes:

Product Backlog

8 Epics

30 User Stories

Sprint 1 Planning

Story Point Estimation

Sprint Backlog

Subtasks and Task Breakdown

Sprint Progress Tracking

<https://saveetha-team-w7mk9z6h.atlassian.net/jira/software/projects/DP/boards/202?filter=&groupBy=none>

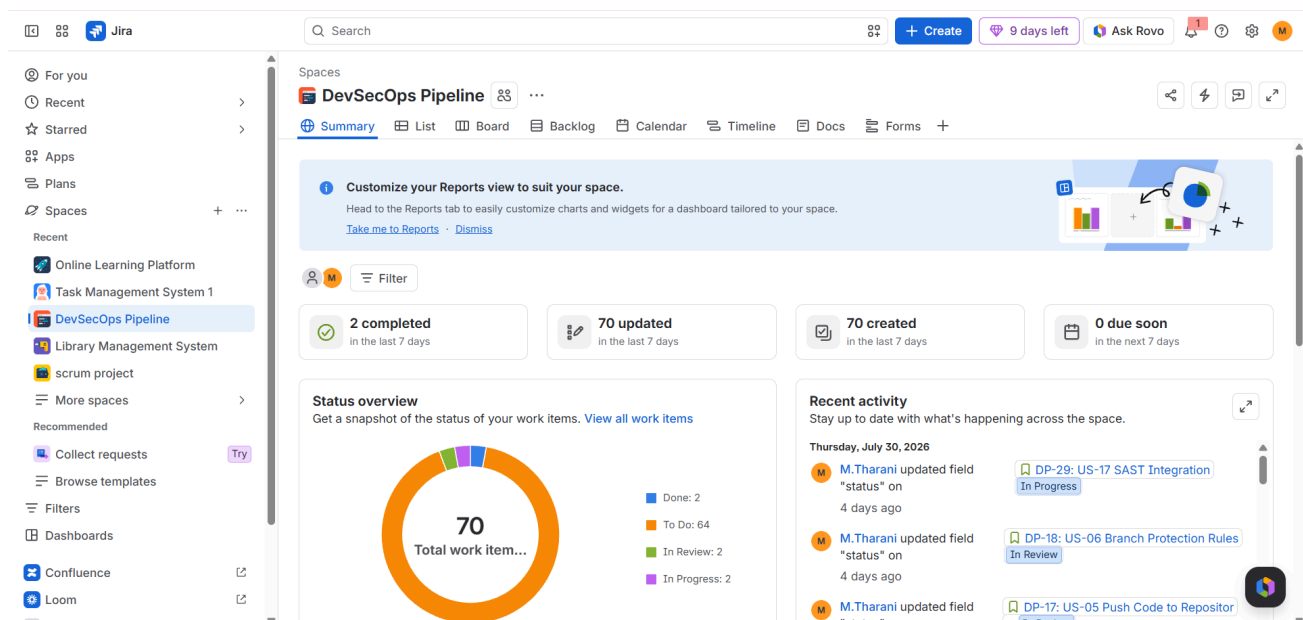


Fig 1:DevSecOps Pipeline Dashboard

saveetha-team-w7mk9z6hatlassian.net/jira/software/projects/DP/boards/202/backlog

Jira

Search

+ Create 9 days left Ask Rovo

Spaces

DevSecOps Pipeline

Summary List Board Backlog Calendar Timeline Docs Forms +

Search backlog Filter

Backlog (27 work items)

91 0 0 Create sprint

DP-1 Secure authentication	To Do	-	=	
DP-2 repository security	To Do	-	=	
DP-3 CI/CD automation	To Do	-	=	
DP-4 vulnerability scanning	To Do	-	=	
DP-5 RBAC	+ Epic To Do	-	=	
DP-15 US-03 Password Reset	EP-01 User Authen...	To Do	3	
DP-16 US-04 Password Complexity Policy	EP-01 User Authen...	To Do	2	
DP-19 US-07 Commit Signing	EP-02 Source Cod...	To Do	3	
DP-20 US-08 Automated Code Review	EP-02 Source Cod...	To Do	5	
DP-23 US-11 Deploy to Staging	EP-03 CI/CD Pipel...	To Do	5	
DP-24 US-12 Pipeline Dashboard	EP-03 CI/CD Pipel...	To Do	3	

jira - Google Search x Jira | Issue & Project Tracking S... x Jira Setup Steps x DevSecOps Pipeline - Backlog x + Ask Gemini - - - - -

saveetha-team-w7mk9z6hatlassian.net/jira/software/projects/DP/boards/202/backlog

Jira

Search

+ Create 13 days left Ask Rovo

Spaces

DevSecOps Pipeline

Summary List Board Backlog Calendar Timeline Docs Forms +

Search backlog Filter

DP Sprint 1 29 Jul - 12 Aug (9 work items)

45 0 0 Complete sprint

DP-13 US-01 Login using Multi-Factor Authentication	EP-01 USER AUTHENT...	TO DO	5	
DP-14 US-02 User Registration	EP-01 USER AUTHENT...	TO DO	3	
DP-17 US-05 Push Code to Repository	EP-02 SOURCE CODE ...	TO DO	3	
DP-18 US-06 Branch Protection Rules	EP-02 SOURCE CODE ...	TO DO	5	
DP-21 US-09 Automated Build Pipeline	EP-03 CI/CD PIPELIN...	TO DO	5	
DP-25 US-13 Container Image Scanning	EP-04 CONTAINER SE...	TO DO	8	
DP-22 US-10 Automated Unit and Integration Testing	EP-03 CI/CD PIPELIN...	TO DO	5	
DP-29 US-17 SAST Integration	EP-05 VULNERABILIT...	TO DO	8	
DP-34 US-21 Role-Based Permission Assignment	EP-06 ROLE BASED A...	TO DO	3	

37°C Mostly cloudy

Search

ENG IN 30% 16:18 30-07-2026

Fig 2: Project Backlog

Create Epic

[Learn about work types](#)

Status
To Do ▾
This is the initial status upon creation

Create from video

Summary *

Summary is required

Description

Improve description

Type /ai to Ask RoVo or @ to mention and notify someone.

☐ Create another Cancel Create

Fig 3:Creating Epics

Jira - Google Search | Jira | Issue & Project Tracking S... | Jira Setup Steps | DevSecOps Pipeline - Backlog | Ask Gemini

saveetha-team-w7/mk9z6h.atlassian.net/jira/software/projects/DP/boards/202/backlog

Jira | Search | + Create | 13 days left | Ask RoVo

Spaces
DevSecOps Pipeline

Summary | List | Board | **Backlog** | Calendar | Timeline | Docs | Forms | +

Search backlog | Filter

DP Sprint 1 29 Jul – 12 Aug (5 work items) | Complete sprint

Item	Status	Priority	Assignee
DP-1 Secure authentication	TO DO	High	Assignee
DP-2 repository security	TO DO	High	Assignee
DP-3 CI/CD automation	TO DO	High	Assignee
DP-4 vulnerability scanning	TO DO	High	Assignee
DP-5 RBAC	TO DO	High	Assignee

+ Create

Backlog (31 work items) | 136 | Create sprint

Item	Status	Priority	Assignee
DP-13 US-01 Login using Multi-Factor Authentication	TO DO	High	Assignee
DP-14 US-02 User Registration	TO DO	High	Assignee
DP-15 US-03 Password Reset	TO DO	High	Assignee

37°C Mostly cloudy | Search | ENG IN | 32% | 16:14 30-07-2026

Fig 4:Creating stories for each epic

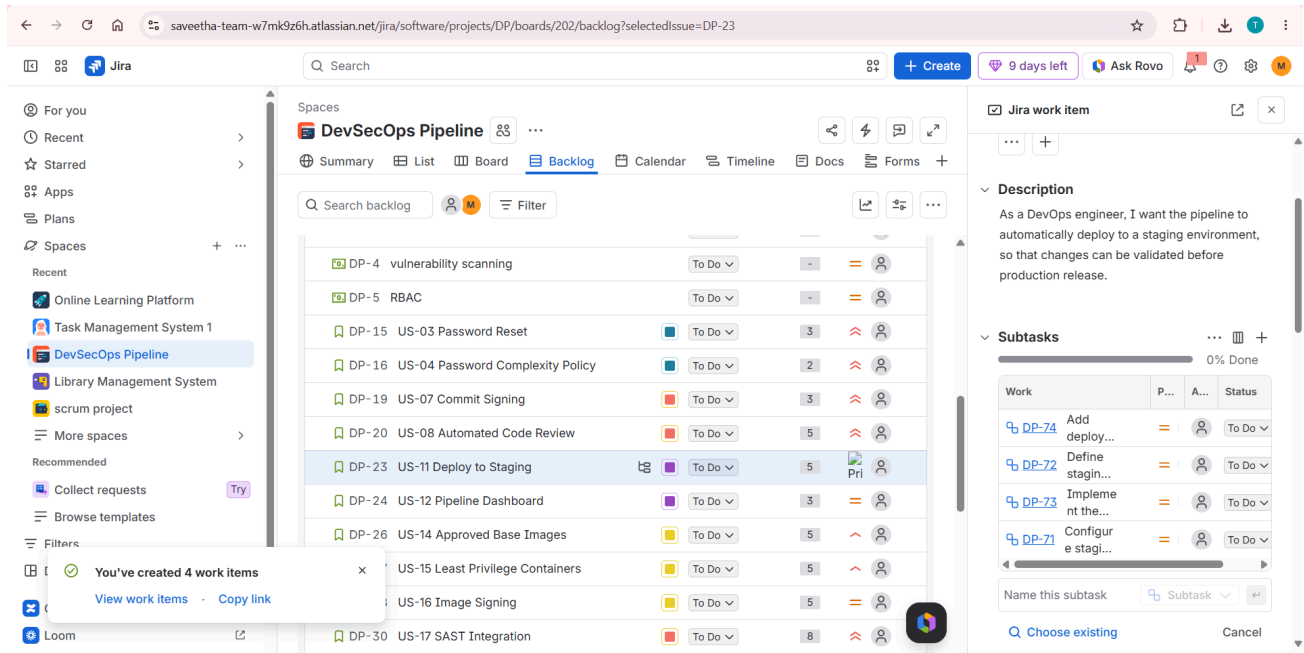


Fig 5: Creating subtasks for user stories

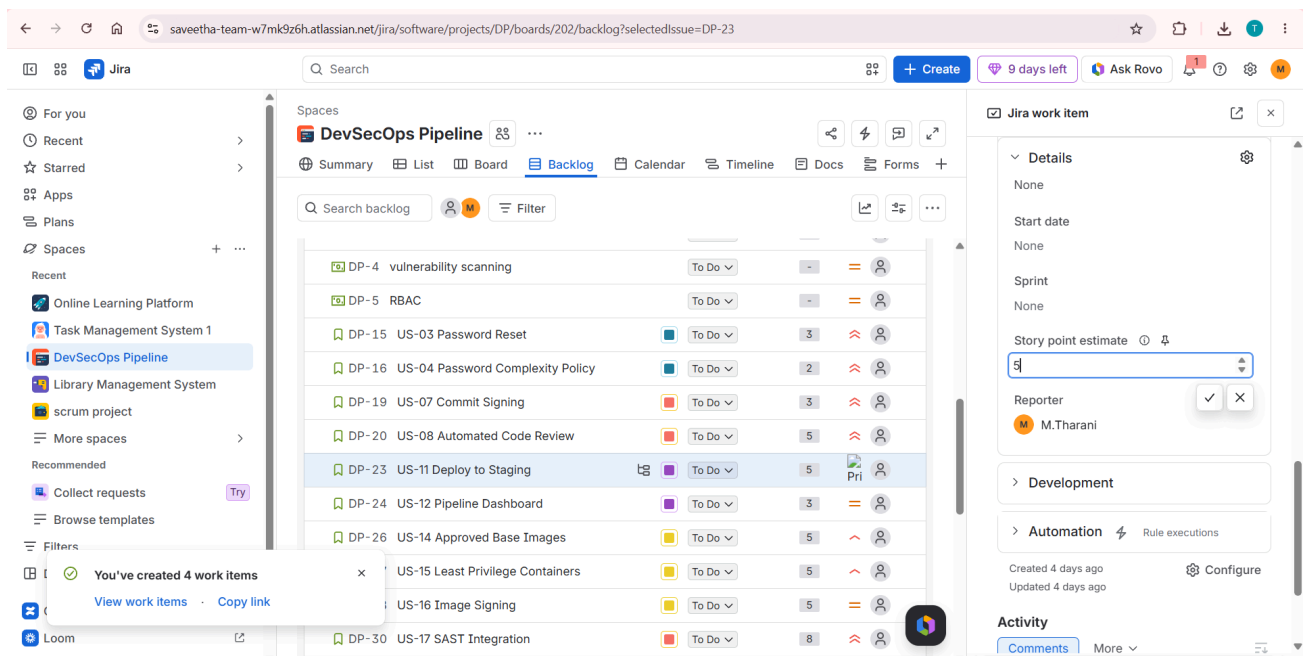


Fig 6: Giving story estimate points

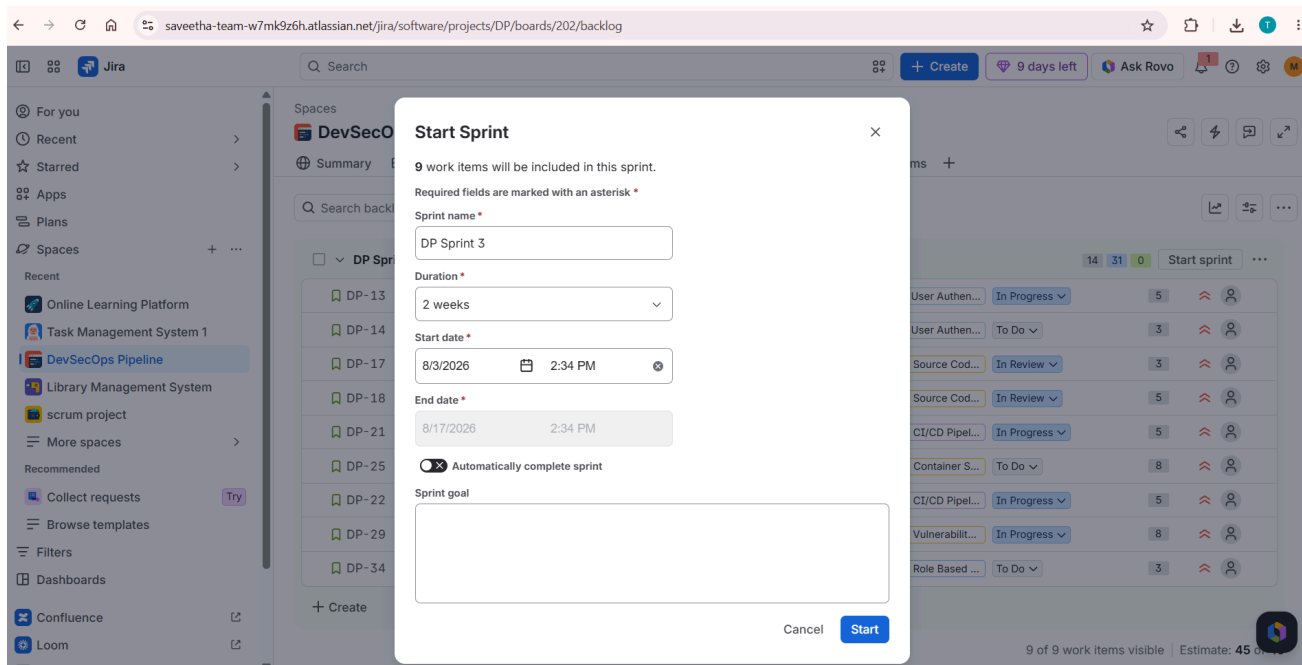


Fig 7:Start sprint

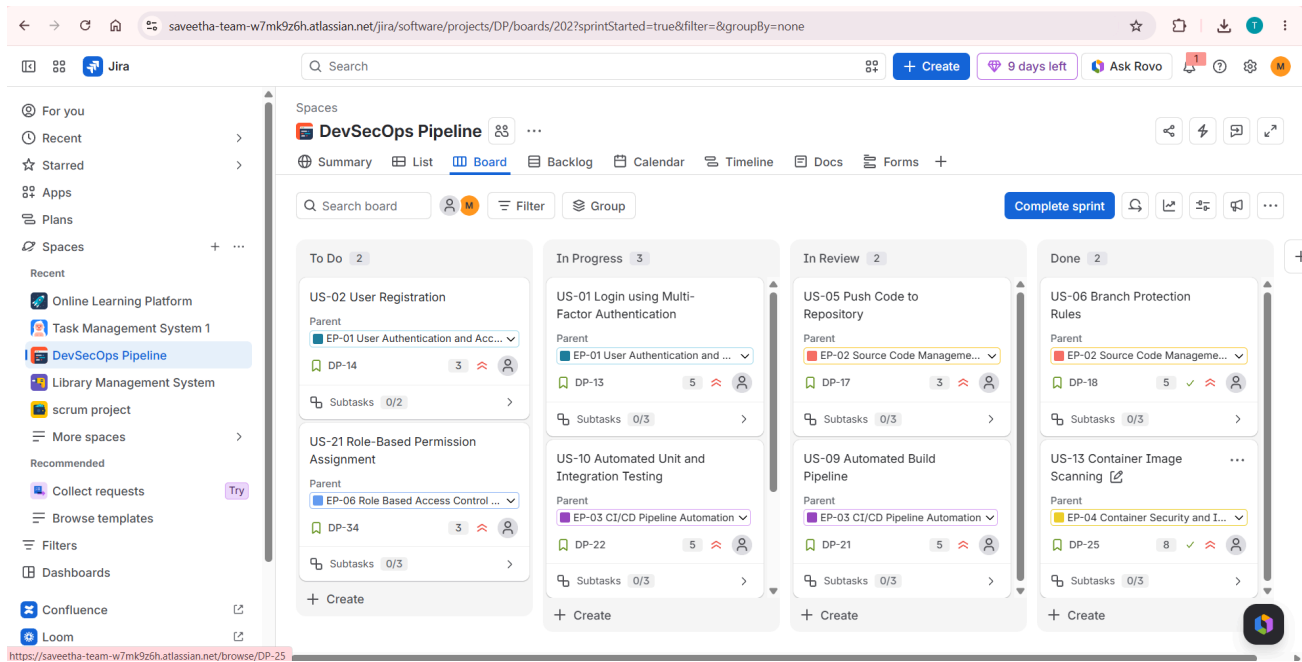


Fig 8:Scrum Board Showing Sprint Progress